

Designing a Web-Scale Crawler

A reference architecture for fetching, parsing, deduplicating and storing on the order of 10 billion web pages, refreshed monthly.

TARGET CORPUS 10e9 pages	REFRESH WINDOW 30 days	SUSTAINED RATE ~4,000 pg/s	RAW STORAGE ~1 PB/snapshot
-----------------------------	---------------------------	-------------------------------	-------------------------------

01 Scope & Requirements

A web crawler (spider) systematically discovers and downloads pages by following links, starting from a seed set. The output feeds a search index, an ML training corpus, or a web archive. The design below assumes a general-purpose, search-engine-scale crawler; the same skeleton scales down cleanly for a vertical crawler by shrinking the frontier and relaxing politeness sharding.

FUNCTIONAL

- Fetch pages from a seed list and follow discovered links (BFS/priority order).
- Respect robots.txt and crawl-delay directives.
- Extract links, text and metadata; hand content to downstream indexers.
- Avoid re-downloading unchanged pages; detect duplicate content.
- Recrawl pages on a freshness schedule proportional to change rate.

NON-FUNCTIONAL

- **Scalable** — horizontal scale to billions of pages.
- **Polite** — never overload an origin server.
- **Robust** — tolerate malformed HTML, spider traps, node failure.
- **Extensible** — new content types / protocols via modules.
- **Fresh** — bounded staleness for high-value pages.

OUT OF SCOPE

Ranking / PageRank computation, the query-serving search index, JavaScript rendering farm (assumed a pluggable fetch backend), and anti-bot evasion. We assume a cooperative-web model with standard politeness.

02 Back-of-the-Envelope

Every downstream decision follows from these numbers, so derive them first and state assumptions explicitly.

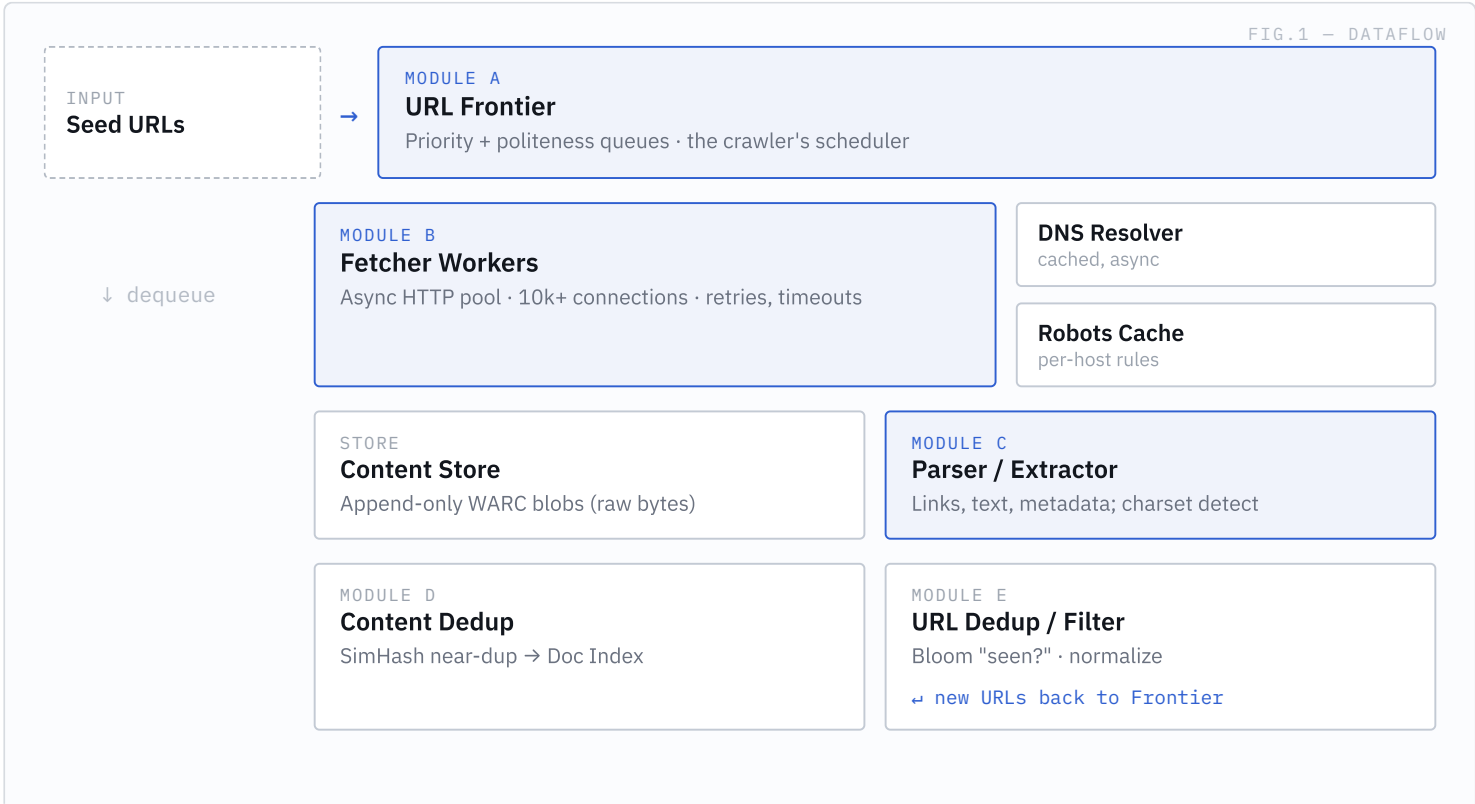
QUANTITY	FORMULA	RESULT
----------	---------	--------

Avg fetch rate	$10e9 / (30 \cdot 86400 \text{ s})$	$\approx 3,858 \text{ pg/s}$
Peak rate (2×)	$2 \cdot \text{avg}$	$\approx 8,000 \text{ pg/s}$
Download bandwidth	$4k \text{ pg/s} \cdot 100 \text{ KB}$	$\approx 400 \text{ MB/s (3.2 Gbps)}$
Raw HTML / snapshot	$10e9 \cdot 100 \text{ KB}$	1 PB (~200 TB gz)
Metadata store	$10e9 \cdot \sim 1 \text{ KB/rec}$	$\approx 10 \text{ TB}$
Seen-URL set	$\sim 100e9 \text{ URLs known}$	Bloom $\approx 120 \text{ GB}$

Key takeaways: (1) 3.2–6.4 Gbps of egress is modest — the crawler is **not** CPU- or bandwidth-bound but **latency- and politeness-bound**, so it lives or dies by concurrency. At ~200 ms/fetch a single connection yields 5 pg/s, so ~800 concurrent fetches sustain 4k pg/s; in practice run tens of thousands of connections across the fleet to absorb slow hosts. (2) The seen-URL set fits in distributed RAM as a Bloom filter, but not as raw strings ($100e9 \cdot 60 \text{ B} \approx 6 \text{ TB}$). (3) Storage is the dominant cost and belongs in an append-only blob store, not a database.

03 High-Level Architecture

The crawler is a loop: the **Frontier** emits URLs, **Fetchers** download them, the **Parser** extracts links and content, new links are deduplicated and fed back into the Frontier, and content is deduplicated and persisted. Shared services (DNS, robots) sit beside the fetch path; a coordination layer shards and heals the fleet.



The feedback edge (Module E → A) makes this a closed loop — the frontier both consumes and produces URLs.

Figure 1. Core crawl loop with shared DNS/robots services beside the fetch path.

04 URL Frontier

The Frontier is the scheduler and the single hardest component. It must satisfy two competing goals simultaneously: **prioritization** (crawl important / stale pages first) and **politeness** (never hit one host too fast). The classic Mercator design solves this with two banks of queues.

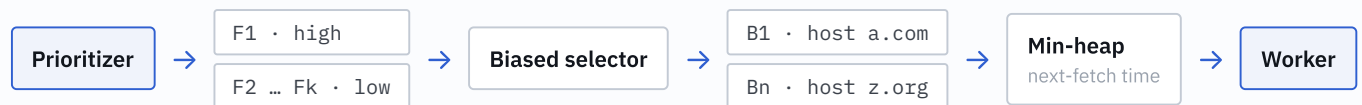
FRONT QUEUES — PRIORITY

A prioritizer assigns each URL a priority $1 \dots k$ (from change rate, PageRank, depth, business value) and routes it to one of k FIFO queues. A biased selector pulls from high-priority queues more often.

BACK QUEUES — POLITENESS

Each back queue holds URLs for **exactly one host**. A min-heap keyed by *next-fetch time* gates release, enforcing per-host crawl-delay. A worker pulls the earliest-ready host, fetches, then reschedules.

FIG.2 — MERCATOR FRONTIER



Persistence & sharding. The frontier is far larger than RAM, so back-queue bodies spill to disk (RocksDB / log-structured files) with only heads cached. It is sharded across nodes by $\text{hash}(\text{host})$ so that **all URLs for a host live on one shard** — this keeps politeness enforcement local and lock-free. A spider-trap guard caps per-host URL count and per-URL depth.

05 Politeness, robots.txt & Rate Limiting

Politeness is a hard constraint, not a nicety — an impolite crawler gets IP-banned and generates abuse complaints. Three mechanisms, layered:

1. **robots.txt** — on first contact with a host, fetch `/robots.txt`, parse allow/disallow rules and Crawl-delay, and cache the parsed ruleset with a TTL (e.g. 24 h). A URL is filtered against this before it ever enters a back queue.
2. **Per-host rate limit** — default one in-flight request per host and a delay of $\max(\text{crawl-delay}, k \cdot \text{RTT})$ between requests. Because a host maps to a single back queue on a single shard, this is enforced by the min-heap timestamp with no cross-node coordination.

3. **Per-IP / subnet limit** — many hosts share an IP (CDNs, shared hosting). Group by resolved IP and also cap concurrent requests per IP so one server behind many vhosts isn't overwhelmed.

INTERVIEW TIP

The single most important politeness insight: **shard the frontier by host**. It collapses distributed rate-limiting into a local scheduling problem. If asked "how do you rate-limit across 1,000 nodes?", the answer is "I don't — a host only ever lives on one node."

06 DNS Resolution & Caching

DNS is a classic hidden bottleneck. A synchronous lookup blocks a fetch thread for tens to hundreds of milliseconds, and public resolvers rate-limit heavy clients. At 4k pg/s a naive design does 4k lookups/s and stalls constantly.

MITIGATIONS

- Local **caching resolver** per node, honoring TTLs; hit rate is very high because a host is fetched many times.
- Run our own **recursive resolvers** to avoid third-party rate limits.
- Make resolution **fully async** so a slow lookup never blocks a worker.
- Pre-resolve and store the IP on the frontier entry so it travels with the URL.

CACHE SIZING

~200 M distinct hosts × ~100 B/entry ≈ **20 GB** — comfortably in RAM. Negative-cache NXDOMAIN too, or dead hosts trigger repeated failing lookups.

```
host → {ip, ttl, resolvedAt}
```

07 Deduplication

Two distinct problems: don't *enqueue* a URL twice (URL dedup), and don't *store* the same content twice (content dedup). They use different tools.

7.1 — URL DEDUP

Normalize first (lowercase host, strip default ports & fragments, sort query params, resolve `./`), then check membership in a **Bloom filter** of seen URL hashes. The Bloom filter gives $O(1)$ checks in ~120 GB instead of 6 TB of raw strings; its one-sided error (false positives only) means we occasionally *skip* a new URL — acceptable — but never re-crawl a seen one incorrectly.

BLOOM FILTER SIZING

$n = 100e9, p = 0.01$

```
# bits per element
m/n = -ln(p) / (ln 2)^2 = 9.585 bits → 100e9 · 9.585 ≈ 120 GB # optimal hash functions
k = (m/n) · ln 2 = 6.64 → round to 7 # tighten to p = 0.001 → 14.4 bits/elt → ~180 GB
```

Shard the filter by `hash(url)` across nodes; each holds a slice in RAM with a disk-backed snapshot for crash recovery. A scalable/partitioned Bloom variant lets it grow as the seen-set does.

7.2 — CONTENT DEDUP

Exact duplicates (mirrors, identical pages on different URLs): hash the normalized body with a strong 64/128-bit digest (e.g. MD5/SHA-1 fingerprint) and check a fingerprint index before storing.

Near-duplicates (same article, different boilerplate/ads): compute a **SimHash** — a locality-sensitive 64-bit fingerprint where similar documents differ in few bits. Two docs are near-dupes if their Hamming distance $\leq \sim 3$. Index SimHashes for fast approximate lookup so only one representative per cluster is indexed.

08 Storage Layer

Split storage by access pattern: bulk immutable bytes go to a blob store; small mutable records go to a wide-column DB; membership structures stay in RAM.

Content store

S3 / HDFS · WARC

Append-only, compressed, immutable raw responses. ~200 TB/snapshot.
Batched into large WARC files, not one object per page.

Metadata store

Bigtable / Cassandra

Per-URL crawl record keyed by URL
hash: status, timestamps, checksum,
blob pointer, next-crawl. ~10 TB.

In-memory sets

Bloom · DNS · robots

Seen-URL Bloom (~120 GB), DNS cache (~20 GB), robots cache — sharded, RAM-resident, disk-snapshotted.

Storing one blob per page would create billions of tiny objects and destroy throughput; instead the fetcher **batches** responses into large sequential WARC files and records a (`file`, `offset`, `length`) pointer in the metadata row. Compression runs ~5:1 on HTML.

09 Data Models & Internal APIs

CRAWL_RECORD (BY URL_HASH)

HOST_RECORD (BY HOST)

```

url_hash      : bytes(16)  # PK
url           : string
host          : string
status        : enum      # ok/err/302
http_code     : int16
content_hash  : bytes(16)
simhash       : uint64
blob_ptr      : (file,off,len)
fetched_at    : ts
next_crawl    : ts
change_freq   : float     # est.
etag          : string

```

```

host          : string    # PK
ip            : bytes
dns_ttl       : int
robots_rules  : blob
crawl_delay   : float
robots_ttl    : ts
last_fetch    : ts
url_count     : int       # trap cap

```

INTERNAL SERVICE RPCS

```

Frontier  enqueue(urls[])          reschedule(url, nextTime)
          dequeueReadyBatch(n) → url[]
Dedup     seenUrl?(urlHash) → bool  markSeen(urlHash)
          contentDup?(simhash) → {dup, clusterId}
DNS        resolve(host) → {ip, ttl}
Robots     allowed?(url) → bool      delayFor(host) → secs
Content    put(bytes) → blob_ptr     get(blob_ptr) → bytes

```

10 Freshness & Recrawl Scheduling

A crawl is never "done" — pages change. Recrawl frequency should be proportional to a page's estimated **change rate**, subject to its importance. A news homepage may recrawl hourly; a static PDF, yearly.

- **Estimate change rate** per URL from observed history (did the content hash change since last fetch?), modeled as a Poisson process; adapt the interval up on no-change and down on change.
- **Cheap change detection** — send If-Modified-Since / If-None-Match (ETag). A 304 Not Modified costs almost nothing and skips the download entirely.
- **Scheduling** — the `next_crawl` timestamp on each record feeds the frontier prioritizer; a sweeper re-enqueues URLs whose time has come.

Tradeoff: freshness vs. politeness/cost. You cannot recrawl everything often. Spend the fixed crawl budget where $\text{importance} \times \text{change_rate}$ is highest — a rank-and-schedule optimization, not a fixed interval.

11 Distributed Coordination & Fault Tolerance

PARTITIONING

The fleet is partitioned by **host** using consistent hashing so a host's frontier, robots rules and rate state are co-located. A coordinator (ZooKeeper / etcd) holds the ring, cluster membership and shard assignment.

FAILURE HANDLING

- Frontier is persisted → a crashed node's shard is reassigned and reloaded from disk.
- In-flight URLs use a checkout/lease with timeout; unacked ones are re-queued (at-least-once).
- Bloom & caches snapshot periodically; rebuildable from the metadata store.
- Poison-URL / trap detection quarantines misbehaving hosts.

The pipeline is decoupled by **durable queues** (Kafka) between fetch → parse → dedup → store, so a slow or failed stage applies backpressure rather than losing work, and each stage scales independently.

12 Bottleneck Analysis & Scaling

BOTTLENECK	SYMPTOM	MITIGATION
DNS	Threads stall on lookups	Async + local cache + own resolvers
Politeness	Few big hosts throttle throughput	More hosts in flight; per-host parallel queues
Frontier I/O	Disk-bound enqueue/dequeue	Log-structured store; batch; cache heads
Storage writes	Too many tiny objects	Batch into large WARC sequential writes
Hot shard	One host / range overloaded	Consistent hashing + rebalancing
Spider traps	Infinite calendars / faceted URLs	Depth + per-host caps; URL heuristics

13 Key Tradeoffs

Coverage vs. freshness

A fixed budget is split between discovering new pages and refreshing known ones. Bias by page importance.

Politeness vs. throughput

Stricter per-host limits protect servers but cap speed on link-dense sites. Widen the host set, don't hammer.

Bloom accuracy vs. RAM

Exactly-once vs. simplicity

Lower false-positive rate costs more bits/element. 1% FP at 120 GB is a sensible default; skipped URLs are cheap.

At-least-once with idempotent stores is far simpler than exactly-once and duplicates are cheaply dedup'd anyway.

END OF DOCUMENT

[Web Crawler](#) · v1.0